COSC490LLMs

Homework 2: Language modeling + count-based LMs + preliminaries to neural networks

Name: Dustin O'Brien
Collaborators, if any: None
Sources used for your homework, if any: Currently None

Homework goals: After completing this homework, you should be able to be comfortable with various foundational concepts related to our course. Specifically, we hope you will gain the followings:

- Get more comfortable with linear algebra, specifically gradients and Jacobians!
- Gain a better grasp of count-based language models!
- Get comfortable with activation functions such as the Softmax function!
- Know how to develop your own classifier!

How to hand in your written work: Via MyClasses as before.

1 Concepts, intuitions and big picture

1. The n-gram models we've shown make the n-th order Markov assumption, i.e. that the distribution of words depends only on the previous $n-1$ words. What properties of language will it not capture? Discuss (briefly) several distinct ways in which this assumption is false for natural language.
   Answer: *The first major flaw of the markov assumptions is that it doesnt consider any added context given for example English status as an SVO language we know that context relating to the object will not be known during the prediction of the verb and similarly context to what the subject is doing and acting upon will not be known in its prediction therefore cannot be considered. A sentence for example the ____ cut down the tree will only have the information the ____ which is unlikely to give sufficient context to predict the word. Another major problem with the markov assumptions is for small n long distance words that provide context will not be considered for example given n=2 with the sentence The logger cut down his own ____ the context for last word would his own ____ which removes valuable context this problem is however minimized by larger n. The last problem is especially with large n is that n grams are unlikely to capture unique and original scenarios which although obviously gramatically correct lack proper data to use*

2. Follow-up to previous question: Despite the Markov assumption, n-gram models are remarkably good at predicting the next word. Discuss why this might be. What information is in the previous word(s) that makes these models perform so surprisingly well? In particular, what kinds of grammatical information do they capture?
   Answer: *N grams do manage to capture basic sentence structure like given an adjective a noun will almost always follow as virtually no example of an english sentence where this doesnt hold thus N-grams can easily capture it. This would significantly beat the performance of random guessing words that won't be able to capture this.*

3. Explain how are perplexity and cross-entropy loss related?
   Answer: *In the formula for perplexity the exponent is simply a version of the cross entropy formula. Since under the assumption that the test sets n-gram probablity matches that of the language as a whole then upon the $\sum_{i=1}^{n}$ will have reoccuring n grams by chaning the summation from $\sum_{i=1}^{n}$ to a set where the sums are instead on a n gram by n gram basis then we create a computationally equivalent problem by simply having some constant c_i where i represents an index of the set of n grams called x the following equation works*

$$\sum_{i=1}^{x} x_i \log\left(q(x_i)\right)$$

   this is simply using the fact that multiplication can replace the multiple additions of the same n-gram in this case the full formula will be

$$-\frac{1}{n}\sum_{i=1}^{x} x_i \log\left(q(x_i)\right) \Rightarrow -\sum_{i=1}^{x} \frac{x_i}{n} \log\left(q(x_i)\right)$$

   and using the previous assumption that the dataset matches that of the language in reality $\frac{x_i}{n} = p(x_i)$ therefore fulfilling the dormuila for cross entropy and since this formula is the exponent it is clearly related to the entire perplexity equation.

4. For a vocabulary of $|V|$ words, what would you expect perplexity to be if your model predictions were completely random? Compute the corresponding cross-entropy loss for $|V| = 2000$ and $|V| = 10000$, and keep this in mind as a baseline.
   Answer: *We know that there exists a $\frac{1}{n}$ chance of getting an answer right randomly guessing given n words. we also have an n of 2000 obviously thus we get the following equation.*

$$H = -\frac{1}{2000}\sum_{i=1}^{2000} \log_2 \frac{1}{2000} \rightarrow \log_2 2000$$

   and since the true perplexity equation is

$$ppl(x) = 2^{\log_2 2000} = 2000$$

   $\therefore$ our final perplexity is a flat 2000 *for a vocabulary of* 2000 *doing the same logic on the second problem of a vocabulary of* 10000 *we will get a perplexity of* 10000

5. Which of these are reasons for the recent wave of neural networks taking off? (check the options that apply.)
   ☒ We have access to a lot more computational power.
   ☐ Neural Networks are a brand new field.

☒ We have access to a lot more data.
☒ There has been significant improvements in benchmarks in speech recognition and NLP.
Answer:

6. What does a neuron compute?
☐ A neuron computes an activation function followed by a linear function ($z = Wx + b$)
☒ A neuron computes a linear function ($z = Wx + b$) followed by an activation function.
☐ A neuron computes a function g that scales the input x linearly ($Wx + b$).
☐ A neuron computes the mean of all features before applying the output to an activation function.
Answer:

7. What is the point of applying a Softmax function to the logits output by a sequence classification model?
☒ It softens the logits so that they're more reliable.
☒ It applies a lower and upper bound so that they're understandable.
☒ The total sum of the output is then 1, resulting in a possible probabilistic interpretation.
Answer:

2 Linear Algebra Recap

2.1 Gradients

Consider the following scalar-valued function:

$$f(x, y, z) = x^2 y + \sin(z + 6y).$$

1. Compute partial derivatives with respect to x, y and z.
   Answer:

$$\frac{\partial f}{\partial x} = 2xy$$
$$\frac{\partial f}{\partial y} = x^2 + 6\cos(z + 6y)$$
$$\frac{\partial f}{\partial z} = \cos(z + 6y)$$

2. We can consider f to be a function that takes a vector $\theta \in \mathbb{R}^3$ as input, where $\theta = [x, y, z]^\top$. Write the gradient as a vector and evaluate it at $\theta = [3, \pi/2, 0]^\top$.
   Answer:

$$\begin{bmatrix} 2xy \\ x^2 + 6\cos(z + 6y) \\ \cos(z + 6y) \end{bmatrix} = \begin{bmatrix} 2(3)(\frac{\pi}{2}) \\ 3^2 + 6\cos\left(0 + 6(\frac{\pi}{2})\right) \\ \cos\left(0 + 6\left(\frac{\pi}{2}\right)\right) \end{bmatrix} = \begin{bmatrix} 3\pi \\ 3 \\ -1 \end{bmatrix}$$

2.2 Gradients of vectors

Let $\mathbf{x}, \mathbf{c} \in \mathbb{R}^n$ and $\mathbf{A} \in \mathbb{R}^{n \times n}$. For the following parts, before taking any derivatives, identify what the derivative looks like (is it a scalar, vector, or matrix?) and how we calculate each term in the derivative. Then carefully solve for an arbitrary entry of the derivative, then stack/arrange all of them to get the final result.

- Show that $\frac{\partial}{\partial \mathbf{x}}\left(\mathbf{x}^\top \mathbf{c}\right) = \mathbf{c}^\top$.
  Answer:

$$\mathbf{xc} = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \begin{bmatrix} c_0 \\ c_1 \\ c_2 \\ \vdots \\ c_n \end{bmatrix} \rightarrow \mathbf{x}^\top \mathbf{c} = \begin{bmatrix} x_0 x_1 x_2 \cdots x_n \end{bmatrix} \begin{bmatrix} c_0 \\ c_1 \\ c_2 \\ \vdots \\ c_n \end{bmatrix} = \begin{bmatrix} c_0 x_0 + c_1 x_1 + c_2 x_2 \cdots c_n x_n \end{bmatrix} \rightarrow$$

$$\frac{\partial}{\partial \mathbf{x}} \begin{bmatrix} c_0 x_0 + c_1 x_1 + c_2 x_2 \cdots c_n x_n \end{bmatrix} = \begin{bmatrix} c_0 c_1 c_2 \cdots c_n \end{bmatrix}$$

- Show that $\frac{\partial}{\partial \mathbf{x}}\left(\|\mathbf{x}\|_2^2\right) = 2\mathbf{x}^\top$.
  Answer:

$$\|\mathbf{x}\|_2^2 = \begin{bmatrix} x_0^2 + x_1^2 + x_2^2 \cdots x_n^2 \end{bmatrix} \rightarrow \frac{\partial}{\partial \mathbf{x}} \begin{bmatrix} x_0^2 + x_1^2 + x_2^2 \cdots x_n^2 \end{bmatrix} = 2x_0 2x_1 2x_2 \vdots 2x_n = 2 \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

- Show that $\frac{\partial}{\partial \mathbf{x}}(\mathbf{Ax}) = \mathbf{A}$.
  Answer:

$$\begin{bmatrix} a_{0,0} & a_{1,0} & a_{2,0} \cdots a_{n,0} \\ a_{0,1} & a_{1,1} & a_{2,1} \cdots a_{n,1} \\ \vdots & \vdots & \ddots \\ a_{0,n} & a_{2,n} & a_{2,n} \cdots a_{n,n} \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} a_{0,0} x_0 + a_{1,0} x_1 + a_{2,0} x_2 \cdots a_{n,0} x_n \\ a_{0,1} x_0 + a_{1,1} x_1 + a_{2,1} x_2 \cdots a_{n,1} x_n \\ \vdots \\ a_{0,n} x_0 + a_{1,n} x_1 + a_{2,n} x_2 \cdots a_{n,n} x_n \end{bmatrix} \rightarrow$$

$$\frac{\partial}{\partial x}\begin{bmatrix} a_{0,0}x_0 + a_{1,0}x_1 + a_{2,0}x_2 \cdots a_{n,0}x_n \\ a_{0,1}x_0 + a_{1,1}x_1 + a_{2,1}x_2 \cdots a_{n,1}x_n \\ \vdots \\ a_{0,n}x_0 + a_{1,n}x_1 + a_{2,n}x_2 \cdots a_{n,n}x_n \end{bmatrix} = \begin{bmatrix} a_{0,0} & a_{1,0} & a_{2,0} \cdots a_{n,0} \\ a_{0,1} & a_{1,1} & a_{2,1} \cdots a_{n,1} \\ \vdots & \vdots & \ddots \\ a_{0,n} & a_{2,n} & a_{2,n} \cdots a_{n,n} \end{bmatrix}$$

- Show that $\frac{\partial}{\partial \mathbf{x}}\left(\mathbf{x}^\top \mathbf{A}\mathbf{x}\right) = \mathbf{x}^\top(\mathbf{A} + \mathbf{A}^\top)$.
  Answer:

$$\frac{\partial}{\partial x}\begin{bmatrix} x_0 & x_1 & \cdots x_n \end{bmatrix}\begin{bmatrix} a_{0,0} & a_{0,1} & \cdots & a_{0,n} \\ a_{1,0} & a_{1,1} & \cdots & a_{1,n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n,0} & a_{n,1} & \cdots & a_{n,n} \end{bmatrix}\begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_n \end{bmatrix} =$$

$$\frac{\partial}{\partial x}\begin{bmatrix} x_0 & x_1 & \cdots x_n \end{bmatrix}\begin{bmatrix} a_{0,0}x_0 + a_{0,1}x_1 \cdots a_{0,n}x_n \\ a_{1,0}x_0 + a_{1,1}x_1 \cdots a_{1,n}x_n \\ \vdots \\ a_{n,0}x_0 + a_{n,1}x_1 \cdots a_{n,n}x_n \end{bmatrix} =$$

$$\frac{\partial}{\partial x}\left[a_{0,0}x_0^2 + a_{0,1}x_0x_1 \cdots a_{0,n}x_0x_n + a_{1,0}x_1x_0 + a_{1,1}x_1^2 \cdots a_{1,n}x_1x_n \cdots a_{0,n}x_0x_n + a_{1,n}x_1x_n + a_{1,n}x_1x_n \cdots a_{n,n}x_n^2\right] =$$

$$\begin{bmatrix} 2a_{0,0}x_0 + a_{0,1}x_1 \cdots a_{0,n}x_n & a_{1,0}x_0 + 2a_{1,1}x_1 \cdots a_{1,n}x_n & \cdots & a_{1,n}x_0 + a_{1,n}x_1 \cdots 2a_{n,n}x_n \end{bmatrix}$$

$$\begin{bmatrix} x_0 & x_1 & \cdots & x_n \end{bmatrix}\left(\begin{bmatrix} a_{0,0} & a_{0,1} & \cdots & a_{0,n} \\ a_{1,0} & a_{1,1} & \cdots & a_{1,n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n,0} & a_{n,1} & \cdots a_{n,n} & \end{bmatrix} + \begin{bmatrix} a_{0,0} & a_{1,0} & \cdots & a_{n,0} \\ a_{0,1} & a_{1,1} & \cdots & a_{n,1} \\ \vdots & \vdots & \ddots & \vdots \\ a_{0,n} & a_{1,n} & \cdots a_{n,n} & \end{bmatrix}\right) =$$

$$\begin{bmatrix} x_0 & x_1 & \cdots & x_n \end{bmatrix}\begin{bmatrix} 2a_{0,0} & a_{1,0} + a_{0,1} & \cdots & a_{n,0} + a_{0,n} \\ a_{0,1} + a_{1,0} & 2a_{1,1} & \cdots & a_{n,1} + a_{1,n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{0,n} + a_{n,0} & a_{1,n} + a_{n,1} & \cdots & 2a_{n,n} \end{bmatrix} =$$

$$\begin{bmatrix} 2a_{0,0}x_0 + a_{0,1}x_1 \cdots a_{0,n}x_n & a_{1,0}x_0 + 2a_{1,1}x_1 \cdots a_{1,n}x_n & \cdots & a_{1,n}x_0 + a_{1,n}x_1 \cdots 2a_{n,n}x_n \end{bmatrix}$$

Since both result in the same vector as output they must be equal

Note: The above equations follow the numerator layout notation which is commonly following in linear algebra, but there are alternative notational conventions as well.

2.3 Jacobian

Consider the following vector function from $\mathbb{R}^3$ to $\mathbb{R}^3$:

$$\mathbf{f}(\boldsymbol{\theta} = [x_1, x_2, x_3]) = \begin{cases} \sin(x_1x_2x_3) \\ \cos(x_2 + x_3) \\ \exp(-\frac{1}{2}x_3^2) \end{cases}$$

1. What is the Jacobian matrix* of $\mathbf{f}(\boldsymbol{\theta})$?
   Answer:

$$\begin{bmatrix} \cos(x_1x_2x_3)x_2x_3 & 0 & 0 \\ \cos(x_1x_2x_3)x_1x_3 & -\sin(x_2 + x_3) & 0 \\ \cos(x_1x_2x_3)x_1x_2 & -\sin(x_2 + x_3) & -x_3\exp\left(\frac{-1}{2}x_3^2\right) \end{bmatrix}$$

*https://en.wikipedia.org/wiki/Jacobian_matrix_and_determinant

2. Evaluate the Jacobian matrix of $\mathbf{f}(\boldsymbol{\theta})$ at $\boldsymbol{\theta} = [1, \pi, 0]$.
   Answer:

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ \pi & 0 & 0 \end{bmatrix}$$

3 n-gram Language Models

3.1 A toy n-gram language model

Consider the following vocabulary, $V = \{\texttt{BOS}, \texttt{EOS}, \texttt{who}, \texttt{framed}, \texttt{roger}, \texttt{rabbit}, \texttt{the}\}$ where BOS is the dummy token indicating the beginning of a sentence, and EOS indicates the end of a sentence. Consider the following training data:

```
BOS who EOS
BOS who framed roger EOS
BOS roger rabbit EOS
BOS who framed roger rabbit EOS
BOS roger framed who EOS
BOS who framed the rabbit EOS
```

1. Compute the following probabilities: $P(\texttt{rabbit})$, $P(\texttt{rabbit}|\texttt{roger})$, $P(\texttt{EOS}|\texttt{rabbit})$.
   Answer:
   P(rabbit) = $\frac{3}{29}$
   P(rabbit | roger) = $\frac{1}{2}$
   P(EOS | rabbit) = 1

2. Briefly explain what the sparsity problem is in n-gram language models?
   Answer: *The sparsity problem is when as the size of a vocabulary grows common among large n the number of instances of a specific n gram happening generally approaches 0 and is zero as there simply exists too many possible permuations of ngrams for test dataset to be sufficient.*

4 Challenges of linear classifiers of sentences

A simple way to compute a representation for a phrase s is to add up the representations of the words in that phrase: $\text{repr}(s) = \sum_{w \in s} v_w$, where $w \in s$ are the word in s and $v_w \in \mathbb{R}^d$ is the embedding for word w.

1. Now, consider sentiment analysis on a phrase in which the predicted sentiments are

$$f(s; \theta) = \theta \cdot \text{repr}(s),$$

for some choice of parameters θ. Note that here $\theta \in \mathbb{R}^d$ and "$\cdot$" is the "dot-product". Prove that in such a model, the following inequality cannot hold for any choice of θ and word embeddings:

$$f(\texttt{good}; \theta) > f(\texttt{not good}; \theta)$$
$$f(\texttt{bad}; \theta) < f(\texttt{not bad}; \theta)$$

Thereby, showing the inadequacy of this model in capturing negations.[†]

Answer: *suppose the embedding value for not $= \boldsymbol{x}$, good $= \boldsymbol{y}$, and bad $= \boldsymbol{z}$. Lets assume the statement $f(good; \theta) > f(not\ good; \theta)$ and $f(bad; \theta) < f(not\ bad; \theta)$ are true.*

Using the definetion of $f(x)$ we know the following.

$$\theta \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_d \end{bmatrix} > \theta \left(\begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_d \end{bmatrix} + \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_d \end{bmatrix} \right)$$

by multiplying both sides by a vectors are $\frac{1}{\theta_i}$ for each element i in θ we get the following

$$\begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_d \end{bmatrix} > \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_d \end{bmatrix} + \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_d \end{bmatrix}$$

Then subtracting $\boldsymbol{y}$ we get the following

$$0 > \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_d \end{bmatrix}$$

Continuing on our second assumption that the following is true

$$\theta \begin{bmatrix} z_0 \\ z_1 \\ \vdots \\ z_d \end{bmatrix} < \theta \left(\begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_d \end{bmatrix} + \begin{bmatrix} z_0 \\ z_1 \\ \vdots \\ z_d \end{bmatrix} \right)$$

We can similarly, multiply by a vector who indexes are all inverse of the cooresponding index in θ

$$\begin{bmatrix} z_0 \\ z_1 \\ \vdots \\ z_d \end{bmatrix} < \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_d \end{bmatrix} + \begin{bmatrix} z_0 \\ z_1 \\ \vdots \\ z_d \end{bmatrix}$$

and subtracting $\boldsymbol{z}$ we get the following

$$0 < \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_d \end{bmatrix}$$

This however contradicts the previously stated fact that both $\boldsymbol{x}$ is less than 0

$\therefore$ both statements cannot be true while using this defintion of sentiment

[†]Question credit: "Introduction to Natural Language Processing" by J. Eisenstein.

2. Consider a slight modification to the previous predictive model:

$$f(s;\theta) = \theta \cdot \mathrm{ReLU}\big(\mathrm{repr}(s)\big),$$

where ReLU (rectified linear function) is defined as:

$$\mathrm{ReLU}(x) = \begin{cases} x & \text{if } x \geq 0 \\ 0 & \text{if } x < 0. \end{cases}$$

Given this choice of predictive function, show that it is possible to satisfy the above inequalities for some choice of θ. **Hint:** Show there exists parameters θ and word embeddings $v_{\texttt{good}}$, $v_{\texttt{bad}}$ and $v_{\texttt{not}}$ that the inequalities are satisfied.

Answer: *Similar to before we can ignore θ by multiplying by vector whos indexes are the inverse of each index in θ However consider the following embeddings, where $x = NOT$, $y = good$ and $z = bad$*

$$x = \begin{bmatrix} 1 \\ -2 \end{bmatrix} y = \begin{bmatrix} 0 \\ 3 \end{bmatrix} z = \begin{bmatrix} 3 \\ -2 \end{bmatrix}$$

in this scenario we get the follwing calculations for good and not good

$$ReLU \begin{bmatrix} 1 \\ -2 \end{bmatrix} + \begin{bmatrix} 0 \\ 3 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix} < \begin{bmatrix} 0 \\ 3 \end{bmatrix}$$

which is the desired response at the same time for bad and not bad we get these outputs

$$ReLU \begin{bmatrix} 1 \\ -2 \end{bmatrix} + \begin{bmatrix} 3 \\ -2 \end{bmatrix} = \begin{bmatrix} 4 \\ 0 \end{bmatrix} > \begin{bmatrix} 3 \\ 0 \end{bmatrix}$$

3. Given the above result, explain (in 1-2 sentence) why the use of neural networks (which have more complexity than linear models)

Answer: *This gives insite into why the neural network is used. In a way buy this we can consider negation as having similar properties to the linearly inseperable functions faced by the single layer perceptron. Where the perceptron fails to make a line that can tell one group of datapoints to another properly the neural network is required because we cannot tell a a datapoint such a not good from its similar good properly.*

5 Softmax function

Remember the Softmax functions from the class:

$$\text{Softmax: } \sigma(\mathbf{z}) \triangleq [\sigma(\mathbf{z})_1, \ldots, \sigma(\mathbf{z})_K] \text{ s.t. } \sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}} \quad \text{for } i = 1, \ldots, K \text{ and } \mathbf{z} = (z_1, \ldots, z_K)$$

1. Prove that Softmax is invariant to constant offsets in the input, i.e., for any input vector $\mathbf{z}$ and any constant c,

$$\sigma(\mathbf{z}) = \sigma(\mathbf{z} + c)$$

Pro tip: We make use of this property in practice to increase the numerical stability of our models. Specifically, using $c = -\max_{i \in \{1 \ldots K\}} z_i$, i.e. subtracting its maximum element from all elements of $\mathbf{z}$ would prevent numerical instability due to large values.

Answer: *testing the hypothesis we can do the following. plugging $z + c$ that into the formula for softmax we get the following*

$$softmax\,(z + c) = \frac{e^{z_i + c}}{\sum_{j=1}^{K} e^{z_j + c}} = \frac{e^{z_i} e^{c}}{\sum_{j=1}^{K} e^{z_j} e^{c}} = \frac{e^{z_i} e^{c}}{e^{c} \sum_{j=1}^{K} e^{z_j}} = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}} = softmax\,(z)$$

2. Softmax maintains the relative order of the elements in $\mathbf{z}$. In particular, show that the largest index is intact after applying Softmax:

$$\arg\max_{i \in \{1 \ldots K\}} z_i = \arg\max_{i \in \{1 \ldots K\}} \sigma(\mathbf{z})_i$$

Answer: *assuming the elements in class z doesn't change than we know that the bottom operation of softmax is unchanging and can be represented as a constant c this turns the equation into*

$$\frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}} = \frac{e^{z_i}}{c}$$

since the e^{z_i} is a constantly increasing function the max of it is the maximum input and since the bottom is a constant and always posistive because e^x is awlays posistive it will equally affect all inputs and max should be unchanged

3. Define the Sigmoid function as follows:

$$\text{Sigmoid: } S(x) \triangleq \frac{1}{1+e^{-x}} = \frac{e^x}{e^x+1} = 1 - S(-x).$$

Prove that Softmax is equivalent to the Sigmoid functions when the number of possible labels is two: $K = 2$. Specifically:

$$\sigma_1([z_1, z_2]) = S(z_1 - z_2)$$

Answer: *plugging in $z_1 - z_2$ we get the following,*

$$S(z_1 - z_2) = \frac{e^{z_1 - z_2}}{e^{z_1 - z_2} + 1} = \frac{e^{z_1 - z_2}}{\frac{e^{z_1}}{e^{z_2}} + \frac{e^{z_2}}{e^{z_2}}} = \frac{e^{z_1}}{e^{z_1} + e^{z_2}} = \frac{e^{z_1}}{\sum_{i=1}^{2} e^{z_i}} = \sigma_1([z_1, z_2])$$

6 Programming

In this programming homework, we will

- build a simple count-based (n-gram) language model.
- implement your own gradient descent optimization for the classifier you built in homework 1.

Skeleton Code and Structure: The code base for this homework can be found at MyClasses/Files under the `hw2` directory. Your task is to fill in the missing parts in the skeleton code, following the requirements, guidance, and tips provided in this pdf and the comments in the corresponding .py files. The code base has the following structure:

- `ngram_lm.py` implements a n-gram language model on a subset of Wikipedia.
- `gradient_descent.py` reuse the sentiment classifier on movie reviews you implemented in homework 1, with additional requirements to implement manual softmax, cross-entropy loss, and gradient updates.
- `main.py` provides the entry point to run your implementations in both `ngram_lm.py` and `backprop.py`.
- `hw2.md` provides instructions on how to setup the environment and run each part of the homework in `main.py`

TODOs — Your tasks include 1) generate plots and/or write short answers based on the results of running the code; 2) fill in the blanks in the skeleton to complete the code. We will explicitly mark these plotting, written answer, and filling-in-the-blank tasks as **TODOs** in the following descriptions, as well as a `# TODO` at the corresponding blank in the code.

TODOs (Copy from your HW1). We are reusing most of the `model.py` from homework 1 as the starting point for the `gradient.py` - you will see in the skeleton that they look very similar. Moreover, in order to make the skeleton complete, for all the `# TODO (Copy from your HW1)`, please fill in the blank below them by copying and pasting the corresponding implementations you wrote for homework 1 (i.e. the corresponding `# TODO` in homework 1.)

Submission: Your submission should contain two parts: 1) plots and short answers under the corresponding questions below; and 2) your completion of the skeleton code base, in a `.zip` file

6.1 Count-based LMs

In the lecture, we have learned language modeling as building probabilistic distribution (marginal and joint) over language. Moreover, we can estimate such distributions (e.g. $\mathbf{P}(X_t|X1, \cdots, X_{t-1})$) by *counting*. For example

$$\mathbf{P}(\text{mat}|\text{the cat sat on the}) \approx \frac{\text{count("the cat sat on the mat")}}{\text{count("the cat sat on the")}}$$

We are going to implement such a count-based LM, i.e. n-gram language model.

6.1.1 Data Loading

We will use (a subset of) the WikiText dataset for building the n-gram LM. The WikiText language modeling dataset is a collection of over 100 million tokens extracted from the set of verified Good and Featured articles on Wikipedia. Spend a few minutes reading a few examples on Huggingface to get a better sense of what this dataset looks like. We will use Huggingface's datasets library to download this dataset locally. For efficiency, we only use the first 1e5 training samples to create our LM.

Read the `load_data` function in `ngram_lm.py` for how we obtain the data.

6.1.2 Preprocessing: Sentence Split and Tokenization

As you have seen, each data sample consists of a Wikipedia paragraph. To perform our n-gram counting, we need first split each paragraph into sentences [‡] and then each sentence into a sequence of tokens (i.e. *tokenization* that we introduced in homework 1). In particular, we will use an algorithm by Philipp Koehn for sentence splitting; and use a *sub-word tokenizer* for tokenization. For now, we are using an existing sub-word tokenizer of an existing model from the Huggingface library, but in the future, we will also build our own tokenizer.

A *Sub-word tokenizer* which, as it should be clear from the name, splits each sentence into units smaller than words, based on their frequency. For example, the word "Potentials" is broken into four sub-words: 'Po', '##ten', '##tial', '##s'', where '##' is a special symbol indicating that the sub-word is in the middle of a word.

Sub-word tokenization might initially seem like a bad idea since we are breaking up the word. But it brings it several benefits.

- Handling Out-of-Vocabulary (OOV) words: Sub-word tokenization allows the representation of rare or unseen words as a combination of sub-words or tokens that have been seen in the training data, thus reducing the OOV problem.
- Improved vocabulary size: It allows for a more compact vocabulary size, reducing the size of the language model, and increasing its efficiency.
- Better language model performance: Sub-word tokenization results in a better representation of morphologically rich languages, where words are formed by combining roots and affixes, leading to improved language model performance.
- Cross-lingual compatibility: Sub-word tokenization is language-agnostic, and models trained on sub-word tokens from one language can be applied to other languages, improving cross-lingual compatibility.

How are sub-tokenizers built? We will delve into that in a few weeks! For now, we will just use them!

Read the `sentence_split_and_tokenize_demo` for how exactly an `wikitext` paragraph is converted into sentences and then tokens.

6.1.3 Build the N-gram LM: Let's Count!

TODOs: read the `create_ngrams` function which iteratively processes each paragraph and counts the n-gram statistics, and completes the following lines under three `# TODO`s:

- tokenize the words in the sentence: that takes in a `sentence` and convert it into `tokens` as a list of tokens with the tokenizer.
  Hint: check out how to do tokenization in sentence_split_and_tokenize_demo
- count n-gram statistics over each list of tokens, record them in
  - `ngrams`: the `Counter` for n-grams count
  - `ngram_context`: the `Counter` for context (i.e. (n-1)-grams) count
  - `next_word_candidates`: the `Dictionary` that keeps track of all possible candidate next tokens given the context, i.e. all tokens that, concatenated with the (n-1)-gram context, constitutes an n-gram that exists in the data.

  for all the `Counter` and `Dictionary`, use `tuple` of n-gram or (n-1)-gram tokens as the keys.
  Hint: also check the corresponding comments in the code for specific requirements
- computes the estimated probability of the next word (token) given the context.
  Hint: use the formula in subsection 6.1, and also the corresponding comments in the code

Finally, the `next_word_pred` returned from the `create_ngrams`, will record for each (n-1) gram context, the most probable next word continuation. For this homework, we will set $n = 3$ so that to build a *trigram* language model.

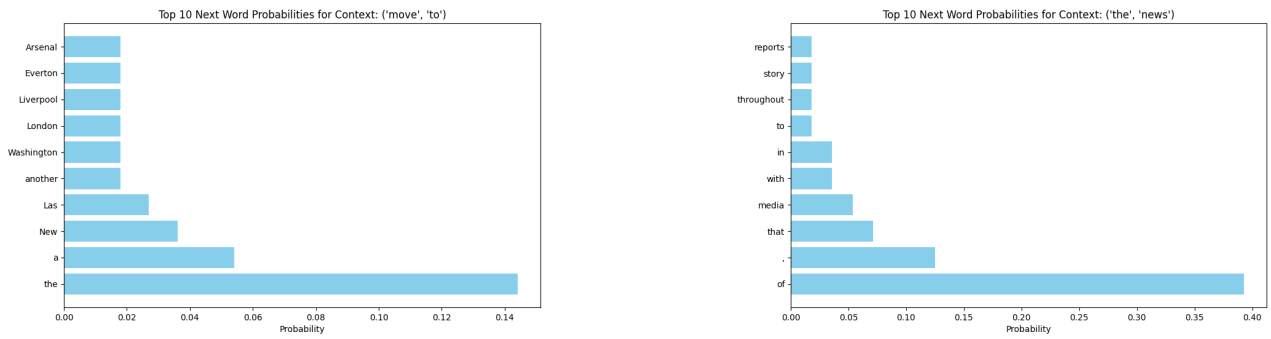
[‡] we count n-gram status within sentence boundary

6.1.4 Visualize the N-gram distribution

TODOs: Complete the `plot_next_word_prob` function in `ngram_lm.py`, and run `run_ngram` in `main.py` to plot the top-10 most probable next token given an input context. Paste the two plots for the two given contexts (provided in `run_ngram`), and describe in 2-3 sentences your findings.
Hints:

- you can check out this tutorial about how to plot bar chart with matplotlib.
- read the comments in the code carefully for more info about the input.

your plots and answer:



(a) top 10 next words of "move to"

(b) top 10 next words of "the news"

Figure 1: The probabilities of top 10 next words of two bi-gram contexts

Answer: *based on the following results we can see that there exists a sort of exponetial distbution tp the data where a significant portion of following n grams are in a couple of tokens while there exists the less popular have roughly the same number of occurences*

6.1.5 Generation: Sample from the LM

Now that we have built our LM, given any prefix[§], we can sample from the LM to generate a full completion. Specifically, at each timestamp, we take the last (n-1) tokens from the prefix as the context and query our LM to get the most probable next token, append it to the prefix, and continue until we sample the stop token our the maximum length is reached.

This sampling procedure is called greedy decoding, as we take the most probable next token each step, there are a few other sampling strategies like beam search (**?**), top-k, and top-p sampling (**?**). Similarly, we will get to them soon.

Read the `generate_text` function in `ngram_lm.py` for details of the generation process described above.

TODOs: Run `run_ngram` in `main.py` and paste the completion of the two given prefixes (provided in `run_ngram`), and describe in 2-3 sentences your findings.
Answer:
Completion 1: According to the report
Completion 2: The president of the association
Findings:
——— generated text 1 ———
According to the report, the first time in the United States, and the other hand, the first time in the United States, and the other hand, the first time
——— generated text 2 ———
The president of the association ' s T̈he One I Love You ' re not here to Connacht side of the game ' s T̈he One I Love You ' re not

The most noticable problem with this text is you can see that it seems loop back around and these loops are basically impossible to break out of given the deterministic nature of n grams and once it repeats it will follow same pattern.

[§]in our case of trigram LM, we require the prefix to have ≥ 2 tokens

6.2 Optimizing the Sentiment Classifier with (Stochastic) Gradient Descent

In the second part of this programming homework, we will revisit the sentiment classifier we built in the last homework. Instead of relying on the PyTorch built-in loss functions, gradient calculations, and weights optimization, we will delve into them and implement our own version!

6.2.1 Reuse Your HW1 Implementation

TODOs (Copy from your HW1): for all the `# TODO (Copy from your HW1)` in `gradient_descent.py`, please fill in the blank below them by copying and pasting the corresponding implementations you wrote for homework 1 (i.e. the corresponding `# TODO` in the `model.py` in homework 1).

6.2.2 Softmax Function

Remember the nn.CrossEntropyLoss we used in homework 1, it can be further decomposed into that 1) normalize the real-value scores of each class (e.g. the logits) into a probability distribution using softmax function, and calculate the cross entropy loss of this probability distribution against the ground truth binary distribution (In practice, PyTorch instead provides the combination of LogSoftmax with negative log likelihood loss). We will first implement the softmax function, which you have been familiar with in the lecture, and section 5.

TODOs: Complete the `softmax` function of the `SentimentClassifier` class in `gradient_descent.py`. Note: you must implement the optimized for numerical stability version described in the **Pro tip** in item 1 of section 5.
Hint: check the comments in the code for specific input-output requirements.
A correct implementation should pass the `test_softmax` in `gradient_descent.py`.

With the softmax function, we turn our neural network into a classifier that assigns a probability distribution over the 2 sentiment classes. Specifically, denoting our input feature vector as $\mathbf{x}$, the `nn.Linear` layer transforms $\mathbf{x}$ into a **logit score** vector $\mathbf{z}$ using a weight matrix W and a bias vector $\mathbf{b}$:

$$\mathbf{z} = \mathbf{x}W + \mathbf{b}$$

This logit score has one element per class, so the weight matrix must have a size (d, c), where c is the number of classes (output labels) and d is the number of dimensions of the input space (features). The bias vector has c elements (one per class).

The logit score is turned into probabilities using the **softmax** operator:

$$\hat{y}_j = \mathbf{P}(\text{class = j}) = \frac{\exp(z_j)}{\sum_k \exp(z_k)}$$

6.2.3 Gradients on Cross Entropy Loss

We will start by defining an objective function that defines "goodness" for our classifier. A common choice for classification is categorical cross-entropy loss or negative log-likelihood.

A discussion or derivation of cross-entropy loss is beyond the scope of this class but a good introduction to it can be found here. A discussion of what makes it superior to MSE for classification can be found here. We will just focus on its properties instead.

Letting y_i denote the ground truth value of class i, and $\hat{y}_i$ be our prediction of class i, the cross-entropy loss is defined as:

$$CE(y, \hat{y}) = -\sum_i y_i \log \hat{y}_i$$

If the number of classes is 2 (which is the case here), we can expand this:

$$CE(y, \hat{y}) = -(y \log(\hat{y}) + (1 - y) \log(1 - \hat{y}))$$

Notice that as our probability for predicting the correct class approaches 1, the cross-entropy approaches 0. For example, if $y = 1$, then as $\hat{y} \to 1$, $CE(y, \hat{y}) \to 0$. If our probability for the correct class approaches 0 (the exact wrong prediction), e.g. if $y = 1$ and $\hat{y} \to 0$, then $CE(y, \hat{y}) \to \infty$.

This is true in the more general M-class cross-entropy loss as well, $CE(y, \hat{y}) = -\sum_i y_i \log \hat{y}_i$, where if our prediction is very close to the true label, then the entropy loss is close to 0, whereas the more dissimilar the prediction is to the true class, the higher it is.
Practical tip: in practice, a very small ϵ is added to the log, e.g. $\log(\hat{y} + \epsilon)$ to avoid $\log 0$ which is undefined.

To optimize this objective, we will compute its gradients with respect to parameters W and $\mathbf{b}$

Before doing that, let's redefine cross-entropy loss in matrix form. With a minibatch of input features $\mathbf{X} = \begin{bmatrix} \mathbf{x_1} \\ \cdots \\ \mathbf{x_N} \end{bmatrix} \in \mathcal{R}^{N \times d}$ where N is the number of instances per batch (batch size), and d is the dimension of feature vectors, and each input $\mathbf{x_j}$ is a row vector of dimension d. And the corresponding output from our network $\hat{\mathbf{Y}} = \begin{bmatrix} \hat{\mathbf{y_1}} \\ \cdots \\ \hat{\mathbf{y_N}} \end{bmatrix} \in \mathcal{R}^{N \times c}$ and label matrix $\mathbf{Y} = \begin{bmatrix} \mathbf{y_1} \\ \cdots \\ \mathbf{y_N} \end{bmatrix} \in \mathcal{R}^{N \times c}$, with row vectors $\mathbf{y}_j \in \{0, 1\}^c$ as one-hot encoding of the class label, and $\hat{\mathbf{y}}_j \in [0, 1]^c$ as the predicted probabilities assigned to each class. Then the loss can be expressed as

$$\mathcal{L}(\mathbf{Y}, \hat{\mathbf{Y}}) = -\frac{1}{N} \sum_j \left[\text{sum}(\mathbf{y}_j \cdot \log \hat{\mathbf{y}}_j) \right] \tag{1}$$

where sum denotes the summation over all elements of the vector, log and $\cdot$ are all element-wise.

After doing the derivations, we obtain the gradient with respect to W and $\mathbf{b}$:

$$\frac{\partial \mathcal{L}}{\partial W} = \frac{1}{N} \sum_i \left[\mathbf{x}_i^\top (\hat{\mathbf{y}}_i - \mathbf{y}_i) \right] \tag{2}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}} = \frac{1}{N} \sum_i \left[\hat{\mathbf{y}}_i - \mathbf{y}_i \right] \tag{3}$$

Verify the correctness of this gradient in your own time! :), or check out this great tutorial for the derivation. Note that because W is a (d, c) matrix, $\frac{\partial \mathcal{L}}{\partial W}$ too. $\mathbf{x}_i^\top (\hat{\mathbf{y}}_i - \mathbf{y}_i)$ is therefore the **outer product** between the error vector $\hat{\mathbf{y}}_i - \mathbf{y}_i$ (c elements) and the input vector $\mathbf{x}$ (d elements).

For more efficiency, we can write the above expression in matrix form:

$$\frac{\partial \mathcal{L}}{\partial W} = \frac{1}{N} \mathbf{X}^\top (\hat{\mathbf{Y}} - \mathbf{Y}) \tag{4}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}} = \frac{1}{N} (\hat{\mathbf{Y}} - \mathbf{Y}) \tag{5}$$

Now, let's put all these together and implement a function for the gradient and loss calculation:

TODOs: Read and complete the missing lines in `gradient_loss` function of the `SentimentClassifier` class in `gradient_descent.py` to calculate the gradients of the weights and biases of the linear layer, as well as the cross entropy loss.
Hint: refer to Equation 4 and Equation 5 for the gradient calculation formulas, and Equation 1 for cross-entropy loss calculation. Also, note that the weight matrix that is stored in `nn.Linear` is actually in the shape of $c \times d$, so you probably need to take a transpose of your gradient calculation.
A correct implementation should pass the `test_gradient_loss` in `gradient_descent.py`.

6.2.4 Gradient Descent

Remember the gradient descent calculation we learned in class, with the gradient and loss calculation we just implemented, we can now optimize our classifier with gradient descent.

TODOs Read and complete the missing lines in `train` function in `gradient_descent.py`, to perform gradient updates on weights and biases, with the given `learning_rate`. Once you finish, run the `single_run_back_prop` in `main.py` to train the model and paste the plot here.
Hint: you can access the weights and biases of the linear layer with `model.linear.weight` and `model.linear.bias`,

respectively.
your plot:

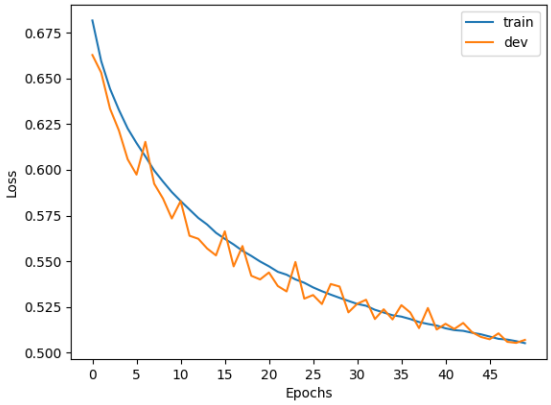


Figure 2: train and dev loss